PROOF OF KEAGS

Posted on 2026.07.20

SUBSCRIBE → EMAIL ATOM RSS

A couple months ago I argued in Formal Vibefication that the advent of LLM code agents fundamentally changed the cost structure of formally verifying software. The main argument I laid out was that, historically, the vast majority of the cost of formal verification was in the proof engineering itself and that this is something that LLMs are particularly well suited to due to the combination of clear success criteria, rich compiler feedback, and proof equivalence.

To test out this thesis, I needed an initial project to which I could apply these techniques. The primary selection criterion for the project was that it would need to be one where having greater certainty over its properties would be useful. Given my background in Bitcoin protocol engineering, I decided that it would be a good candidate and that there would be significant value in formalizing the protocol.

The main reason for applying these techniques to Bitcoin specifically is that, at the time of this writing, the system carries a market capitalization upwards of $1T USD, giving failures in

Secondly, due to its decentralized nature, the progress of protocol development is primarily contingent on being able to present convincing arguments to other developers, as well as the general public, that the changes proposed are both safe and useful.

Finally, to effectively bust myths about what Bitcoin is or is not, it is necessary to clearly state the properties it promises its users. Applying formal verification techniques to Bitcoin has the potential to offer a living body of knowledge about the protocol and machine-checkable evidence for the claims it makes.

However, there's a deeper reason to point this machinery at Bitcoin specifically: it's aligned with Bitcoin's ethos. There's an oft-quoted mandate in Bitcoin development culture:

"Don't Trust. Verify."

This mandate implores us to anchor our belief in the system in work we have done independently rather than work we delegate to others. It is this insistence on independently verifying the correctness of any truth claim that is the bedrock of Bitcoin's

trustworthiness, which therefore underpins the value of Bitcoin itself.

Bitcoin software arrives at the true state of the shared ledger of coin ownership. Rather than trusting an institution to honestly and consistently update the ledger and attest to its state, the software rederives that state independently by following a consistent set of rules, ensuring that each transaction and block adheres to the requirements of the system, and then applies the encoded state transitions until it arrives at the present.

Beyond the operating procedure of the software itself, though, this maxim also applies at another layer. Bitcoin Core and other implementations of the Bitcoin protocol may reclaim the responsibility of operationally verifying the ledger integrity, but on their own they simply move the trust point to the software authors.

For this reason, any respectable implementation of Bitcoin must be open source. For if you are simply running software that others wrote, you are implicitly trusting that their authorship is free of any malice or incompetence that may compromise the integrity of the system's operation. Not only must it be open source, but a serious user of Bitcoin ought to, at some point, make an attempt to actually read at least some of the source code of the implementation that they run.

While this practice does not realistically scale to every user who depends on the system, it is a significantly sturdier foundation upon which to base the claims of Bitcoin's security

The project I am introducing today, btc-verified, takes this philosophy up another layer. If the design choices of Bitcoin serve to verify its operation, and its open-source nature serves to verify its implementation, btc-verified aims to verify its *design*.

It is the same verification maxim applied to Bitcoin's protocol development process itself. This project has a long road ahead of it, but it is not simply an idea. As I write this post today, btc-verified has a specification of Bitcoin's protocol syntax (the data structure hierarchy that is transmitted over the wire), and machine-checked proofs of its serialization laws, anchored against real mainnet blocks and transactions. In future posts I will go over, in detail, the contents of the repo, but today I want to talk about the project conceptually.

For the last 17 years of Bitcoin's operation, the process for making changes to the Bitcoin protocol has been an informal legislative process where anyone in the world can offer proposals, critiques, defenses, appeals, and code to the rest of the user base in service of its continuous improvement. When the dust settles on the ensuing debates, the protocol either changes or it doesn't. This messy process of "technical

consensus″ is important for resisting capture by privileged authorities, but it is not without its pain points.

constituency of participants is a revolving door of volunteers, questions of Bitcoin's design are constantly re-litigated on the mailing list and forums in a way that nearly guarantees an unsatisfying discussion from the point of view of at least one of the participants.

If an inexperienced contributor asks a question or offers a proposal that has been brought up before, they are either denied a satisfying explanation, or an expert in the details of the last time this discussion was settled is required to re-explain why a particular choice was made or why some aspect of the protocol's design cannot be changed.

In rare cases, it's possible to cite the previous discussion in a way that does not dismiss the current inquirer, but even in those cases, the concrete justification isn't often in plain view, but rather threaded through a long back-and-forth between many different people. In an ideal world, it would be possible to cite a durable artifact that supports a senior's concise response to the junior's question or proposal in a way that respects their desire to participate without burdening the senior with the attentional expense of fielding every inquiry.

Secondly, there is a tricky question about what actually defines the Bitcoin protocol. Unlike protocols defined by the IETF, there is no central specification that says what Bitcoin is. In

Bitcoin Core's codebase defines what the Bitcoin protocol is.

While this makes intuitive sense, it is an unfortunate tautology that offers little guidance when we try to make changes to the implementation that don't affect the protocol semantics, or if we try and change the protocol itself to more closely deliver the commonly expected promises that Bitcoin makes. For this it would be useful if we had a purely abstract protocol description that dispensed with as many computational logistics details as possible, and distilled the essential requirements.

With this in hand, we can show how an implementation satisfies those requirements or show the equivalence of two implementations with respect to those requirements. This could then establish that the implementations agree on all specified consensus behavior, making them far safer to run.

Additionally, since any two versions of the same implementation are also effectively different implementations, a formal description of the protocol also gives us a tool that allows us to do impact analysis where proposed changes would be translated into a set of explicit proof obligations. For a proposed change,

existing invariants may remain unaffected, require new proofs, or cease to hold under the revised design.

This is because in the event that the formalization diverges from what the network does in practice, it's actually the *formalization* that's wrong. At some hypothetical point in the future, with an abundance of implementations that all target this spec, that reality may change, but this is not a belief worth entertaining for the foreseeable future.

Thirdly, btc-verified can start to document claims about the protocol that are provably true that may not be obvious from reading the source code, contributing to a growing depth of understanding of how the protocol works. What's more is that these claims and proofs compose in a way where each proven claim becomes a usable premise for the next one. This gives us a repository with compounding proof capital, driving down the cost of analysis on new questions over time.

This is far more ambitious and requires a great deal of care in the discipline of its creation, but it's something that the existing Bitcoin Core codebase and development forums are structurally incapable of offering due to the limitations of both C++ and the English language which is the primary medium of debate.

That said, even as we start to build this up it is extremely important to remember that the theorems only cover exactly what is stated and no more. It can be tempting to overgeneralize

not been proven in the project, then you cannot say that it has been verified.

Now, after discussing all of the promises of the project, it is worth turning our attention to what limitations we still have even with a perfectly executed formal verification of the Bitcoin protocol.

Suppose we got to a complete description of the Bitcoin protocol in Lean (the proof assistant used in btc-verified), does this mean Bitcoin Core is bug-free?

No.

Beyond the protocol description and verification of various theorems against that description, we still would need to connect that specification to how Bitcoin Core (or whatever alternative implementation you prefer) actually implements it for any guarantees to translate to the running software.

The only other way to easily achieve this would be to use Lean's C extraction facility and run that code directly. This would remove a separate handwritten implementation from the correspondence problem, although the resulting executable

would still depend on the correctness of Lean's compiler, runtime, and native toolchain. While this closes the safety gap substantially with a significantly reduced effort, it bears

Secondly, there are a number of design decisions that are of critical importance to Bitcoin's implementation that resist clean descriptions with regard to resource consumption. It is important to the practical security of Bitcoin that the nodes verifying transactions and blocks cannot easily have their computational resources exhausted (memory, CPU, disk, network bandwidth, etc.).

Verifying this would require being able to state theorems about resource consumption. Similar to the observation above about the efficiency of the Lean code, there exist a number of formally correct implementations that can be easily attacked in practice because of concerns over resource usage.

Now, it is possible to state and prove claims regarding the upper bounds of that resource usage, provided that there is a cost model, but cost models are a separate piece of infrastructure that requires careful thought and even asymptotic bounds may not be sufficient for practical claims about resource usage if the constant factors are large.

Thirdly, there are a number of arguments and assumptions about the deployment context of Bitcoin that underpin its safety that are extrinsic to the code actually running. While these

formalization will take substantial effort that does not in any way overlap with the work of formalizing the protocol itself.

Some of Bitcoin's security properties such as reorg resistance and liveness depend on assumptions about the network that cannot be empirically validated in practice. So we might be able to model a network with certain propagation properties and state probabilistic security claims over that network, but there is often no test we could run that ensures the real world network behaves like our model does.

Even if we can measure it empirically to behave the way a simulation based on our model would behave, that doesn't constitute a guaranteed equivalence in a continuously changing environment.

That doesn't make the exercise useless. It would still improve our understanding of the assumptions we are holding when making probabilistic claims over Bitcoin's security, but it does not automatically prove "Bitcoin is secure".

Finally, in some cases it may not even be possible to specify certain game theoretic properties because there are certain

important concepts that cannot be fully enumerated inductively. A clear example of this is that it used to be believed that honest mining is incentive-compatible so long as the miners have less

an arbitrary block timing release policy that defies a tight description.

Disappointing as this limitation may sound, it still ends up being useful because it exposes the boundary of the model itself and identifies where we need to supplement it with empirical evidence, cryptographic assumptions, or good old-fashioned technical judgement. In the Selfish Mining example, had someone set out to prove the incentive compatibility of honest mining, they might very well have discovered this, which leads me to my next point.

Even if we can't formalize all of the important properties we might care about, partitioning issues into those that can be cleanly discharged and those that require something fuzzier can more efficiently direct the extremely scarce developer expertise to reason about and judge designs that can't be easily automated away.

In summary, btc-verified cannot replace the technical consensus process, appoint a formal specification as Bitcoin's authority, or prove Bitcoin "secure" in a single theorem. What it can do, though, is produce a machine-checkable mapping between the

rules and assumptions on which Bitcoin relies and the consequences that follow thereafter.

changes, comparing implementations, and identifying the boundaries where empirical evidence or technical judgement is still required.

Because each proof can be used as a premise for the next, the cost of answering new questions should fall as the body of work grows, and because it lives in a single coherent knowledge base, we preserve a lot more institutional knowledge in the presence of a rotating set of contributors.

This is what it means to take “Don’t Trust. Verify.” seriously at the level of protocol design: make truth claims about the protocol explicit and reduce the claims that must be informally argued one proof at a time.

In future posts I’ll dig into the content of this project as well as talk about the techniques and wisdom I learn in the process. In the next post I’ll give a tour of what btc-verified proves today and how we build up the protocol from its most atomic structures.

New research posts, delivered by email: subscribe.

